

```
1 import pandas as pd
2
3 files = [
4     r"/Users/harp/Desktop/Capstone project/Dataset/2020_01.csv",
5     r"/Users/harp/Desktop/Capstone project/Dataset/2020_02.csv",
6     r"/Users/harp/Desktop/Capstone project/Dataset/2020_03.csv",
7     r"/Users/harp/Desktop/Capstone project/Dataset/2020_04.csv",
8     r"/Users/harp/Desktop/Capstone project/Dataset/2020_05.csv"
9 ]
10
11 dfs = [pd.read_csv(file) for file in files]
12
13 OTP_df = pd.concat(dfs, ignore_index=True)
14
15 print("Merged Dataset Shape:", OTP_df.shape)
16
17 OTP_df.head()
```

Merged Dataset Shape: (965568, 13)

	date	train_id	stop_sequence	from	from_id	to	to_id	scheduled_time	actual_time	delay_minutes	status	line	type
0	2020-01-01	0539	1.0	Hoboken	63.0	Hoboken	63.0	2020-01-01 20:08:00	2020-01-01 20:09:11	1.183333	departed	Montclair-Boonton	NJ Transit
1	2020-01-01	0539	2.0	Hoboken	63.0	Newark Broad Street	106.0	2020-01-01 20:26:00	2020-01-01 20:31:03	5.050000	departed	Montclair-Boonton	NJ Transit
2	2020-01-01	0539	3.0	Newark Broad Street	106.0	Watsessing Avenue	154.0	2020-01-01 20:38:00	2020-01-01 20:40:26	2.433333	departed	Montclair-Boonton	NJ Transit
3	2020-01-01	0539	4.0	Watsessing Avenue	154.0	Bloomfield	19.0	2020-01-01 20:40:00	2020-01-01 20:42:04	2.066667	departed	Montclair-Boonton	NJ Transit
4	2020-01-01	0539	5.0	Bloomfield	19.0	Glen Ridge	50.0	2020-01-01 20:43:00	2020-01-01 20:46:12	3.200000	departed	Montclair-Boonton	NJ Transit

Data from Jan - May 2020 were combined into a single dataset using pd.Concat() With each row representing the train stop event containing scheduled and actual operating times, station information, operator and delay duration

```
1 OTP_df.to_csv("OTP_df.csv", index=False)
2
3 print("Merged file saved as merged_dataset.csv")
```

Merged file saved as merged_dataset.csv

```
1 #Inspecting the Dataset
2 OTP_df.describe()
```

	stop_sequence	from_id	to_id	delay_minutes
count	870025.000000	965565.000000	965565.000000	870025.000000
mean	8.058393	4208.960811	4218.754018	3.761713
std	5.053450	11793.927215	11805.239950	8.532076
min	1.000000	1.000000	1.000000	0.000000
25%	4.000000	52.000000	52.000000	0.366667
50%	7.000000	103.000000	103.000000	2.083333
75%	11.000000	136.000000	136.000000	4.066667
max	26.000000	43599.000000	43599.000000	406.000000

```
1 #For categorical
2 OTP_df.describe(include='object')
```

	date	train_id	from	to	scheduled_time	actual_time	status	line	type
count	965568	965568	965568	965568	870025	965568	965568	965568	965568
unique	139	1545	165	165	172401	653935	3	30	2
top	2020-01-17	0070	New York Penn Station	New York Penn Station	2020-03-04 18:04:00	2020-05-04 05:37:11	departed	Morristown Line	NJ Transit
freq	9785	2100	38932	39142	19	28	870713	153241	924898

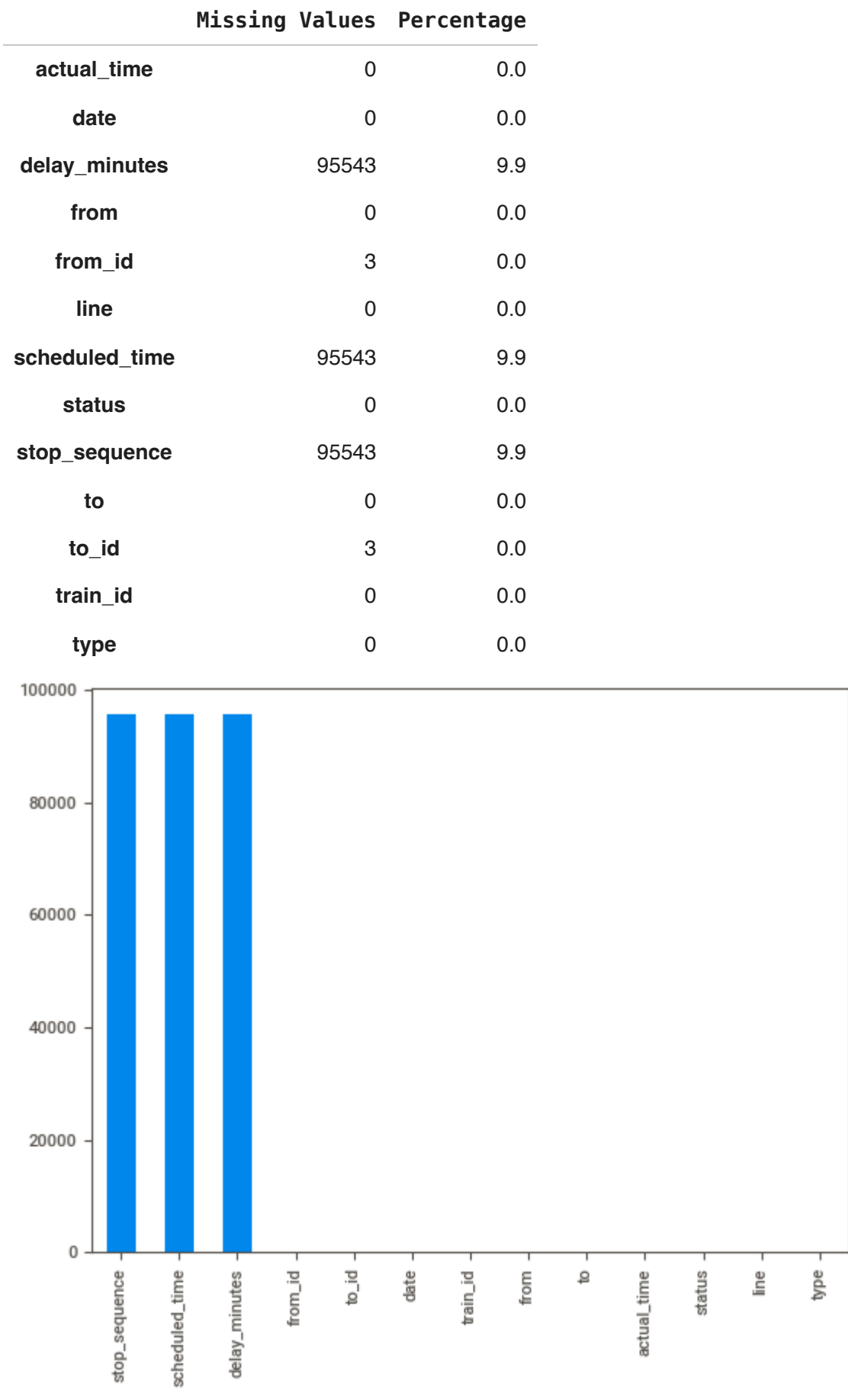
```
1 """# Install (run once)
2 !pip install -q ydata-profiling sweetviz
3
4 # Import
5 from ydata_profiling import ProfileReport
6
7 # Create report
8 profile = ProfileReport(
9     OTP_df,
10     title="Merged Dataset Profiling Report",
11     explorative=True
12 )
13
14 # Save report
15 profile.to_file("ydata_profile_report.html")
16
17 print("Report saved successfully as 'ydata_profile_report.html'")"""
```

Install (run once)!pip install -q ydata-profiling sweetviz\n\n# Import\nfrom ydata_profiling import ProfileReport\n\n# Create report\nprofile = ProfileReport(\n OTP_df,\n title="Merged Dataset Profiling Report",\n explorative=True\n)\n\n# Save report\nprofile.to_file("ydata_profile_report.html")\n\nprint("Report saved successfully as 'ydata_profile_report.html'")

```
1 import sweetviz as sv
2 report = sv.analyze(OTP_df)
3 report.show_html( "sweetviz_report.html")
4 print("Sweetviz report generated.")
5
6 #To download the sweetviz report
7 #files.download("sweetviz_report.html")
```

Report sweetviz_report.html was generated! NOTEBOOK/COLAB USERS: the web browser MAY not pop up, regardless, the report IS saved in your notebook/colab files. Sweetviz report generated.

```
1 #Initial Data quality
2 missing = OTP_df.isnull().sum()
3 missing = (missing.sort_values(ascending=False))
4
5 #Visual representation
6 missing.plot.bar()
7
8 #Missing Percentage
9 missing_percent = OTP_df.isnull().mean()*100
10
11 missing_table = pd.DataFrame({
12     "Missing Values":missing,
13     "Percentage":missing_percent.round(2)
14 })
15
16 missing_table
```



```
1 #Duplicate rows check
2 duplicates= OTP_df.duplicated().sum()
3 duplicates
4 #Removing the duplicate rows
5 OTP_df.drop_duplicates(inplace=True)
```

Duplicate observations were removed, if any present in the dataset.

```
1 #Data Consistency Check
2 OTP_df["type"].unique()
3 OTP_df["line"].unique()
4 categorical = [
5     "status",
6     "line",
7     "type",
8     "from",
9     "to"
10 ]
11
12 for col in categorical:
13
14     print("\n",col)
15
16     print(OTP_df[col].nunique())
17
18     print(OTP_df[col].unique()[:15])
```

status
3
['departed' 'estimated' 'cancelled']

line
30
['Montclair-Boonton' 'KEYSTONE' 'Bergen Co. Line ' 'PENNSYLVANIAN'
'No Jersey Coast' 'Amtrak' 'Main Line' 'CRESCENT' 'REGIONAL' 'CAROLINIAN'
'Morristown Line' 'Northeast Corrdr' 'Raritan Valley' 'Princeton Shuttle'
'ACELA EXPRESS']

type
2
['NJ Transit' 'Amtrak']

from
165
['Hoboken' 'Newark Broad Street' 'Watsessing Avenue' 'Bloomfield'
'Glen Ridge' 'New York Penn Station' 'Newark Penn Station' 'Trenton'
'Waldwick' 'Ridgewood' 'Glen Rock Boro Hall' 'Radburn Fair Lawn'
'Broadway Fair Lawn' 'Plauderville' 'Garfield']

to
165
['Hoboken' 'Newark Broad Street' 'Watsessing Avenue' 'Bloomfield'
'Glen Ridge' 'Bay Street' 'New York Penn Station' 'Newark Penn Station'
'Trenton' 'Philadelphia' 'Waldwick' 'Ridgewood' 'Glen Rock Boro Hall'
'Radburn Fair Lawn' 'Broadway Fair Lawn']

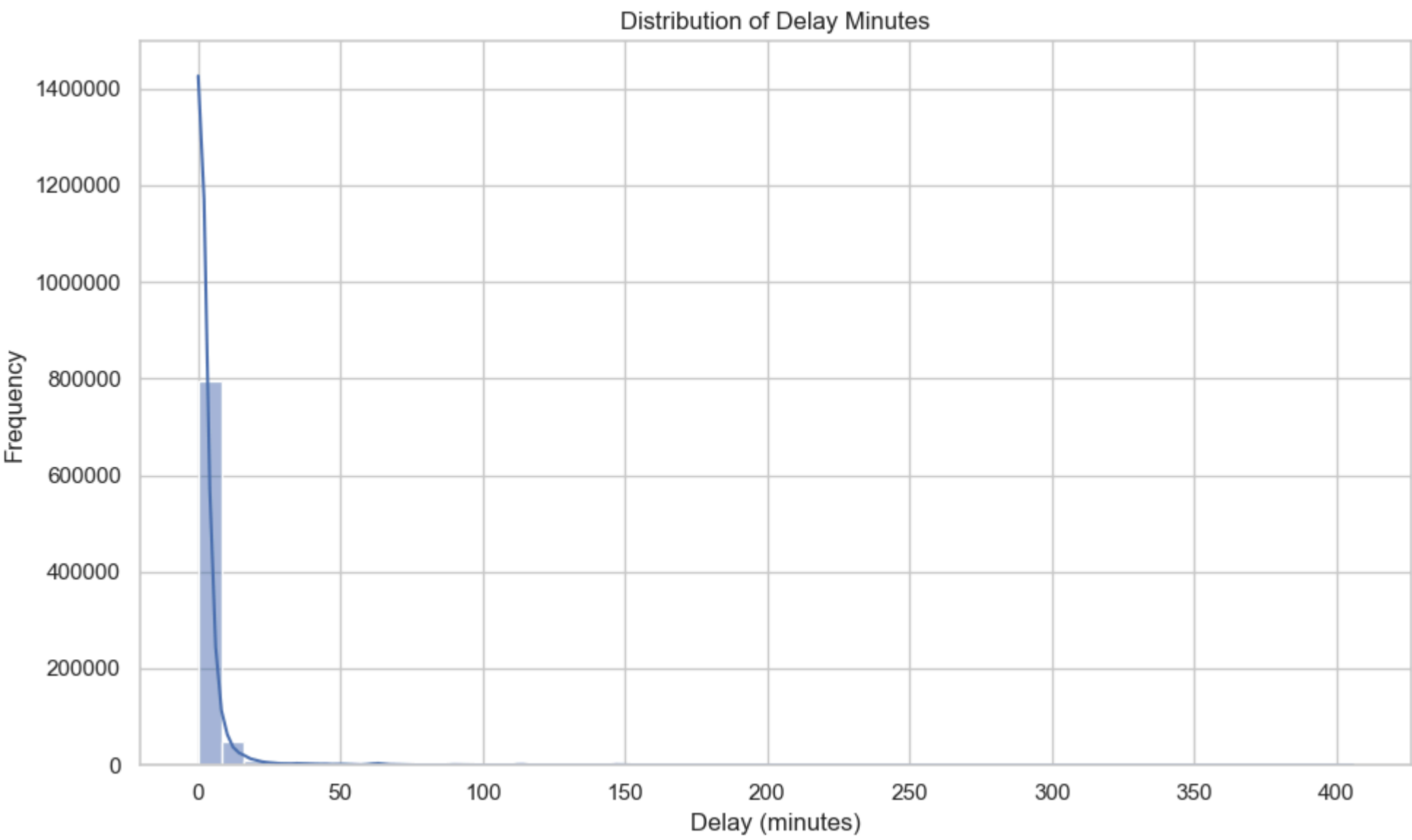
7/25/26, 3:39 PM

1 #TIMESTAMP-PARSING: Convert time columns
2 OTP_df['scheduled_time'] = pd.to_datetime(OTP_df['scheduled_time'])
3 OTP_df['actual_time'] = pd.to_datetime(OTP_df['actual_time'])
4 OTP_df['date'] = pd.to_datetime(OTP_df['date'])
5
6 OTP_df.dtypes

date datetime64[ns]
train_id object
stop_sequence float64
from object
from_id float64
to object
to_id float64
scheduled_time datetime64[ns]
actual_time datetime64[ns]
delay_minutes float64
status object
line object
type object
dtype: object

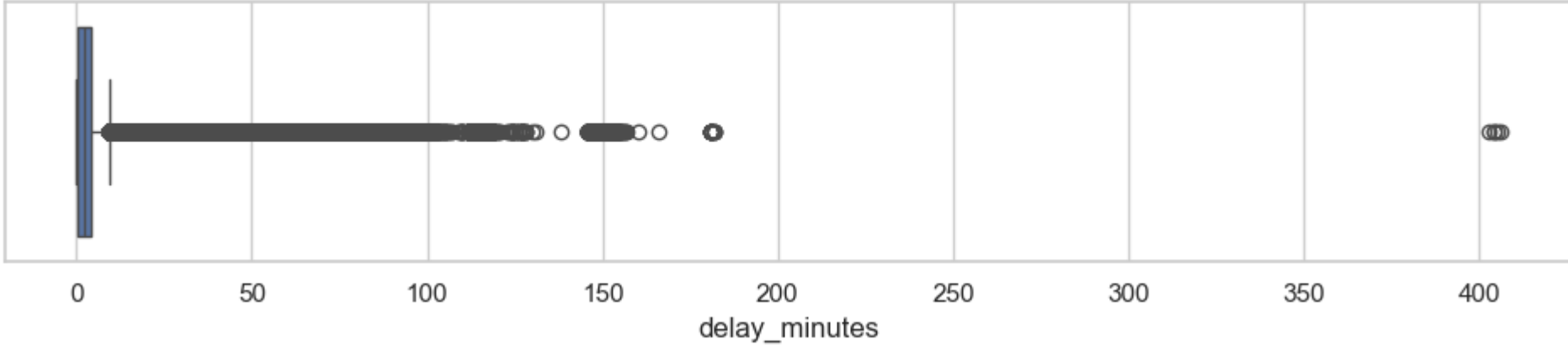
Timestamp Variable were converted to datetime format to enable extraction of temporal features including hour, weekday and month.

```
1 # Display plots inside the notebook
2 %matplotlib inline
3
4 import matplotlib.pyplot as plt
5 import seaborn as sns
6
7 # Set plot style (optional)
8 sns.set_theme(style="whitegrid")
9
10 # Delay Distribution
11 plt.figure(figsize=(10, 6))
12
13 sns.histplot(
14     OTP_df["delay_minutes"].dropna(), # Remove missing values
15     bins=50,
16     kde=True
17 )
18
19 plt.title("Distribution of Delay Minutes")
20 plt.xlabel("Delay (minutes)")
21 plt.ylabel("Frequency")
22
23 plt.tight_layout()
24 plt.show()
25
```



Delay minutes exhibit a positively skewed distribution with a long right tail, indicating that while most train experience minimal delays, a small number experience substantial operational disruptions. This supports the use of median imputation and robust statistical methods.

```
1 #Boxplot
2 plt.figure(figsize=(12,2))
3
4 sns.boxplot(
5     x=OTP_df["delay_minutes"]
6 )
7
8 plt.show()
```



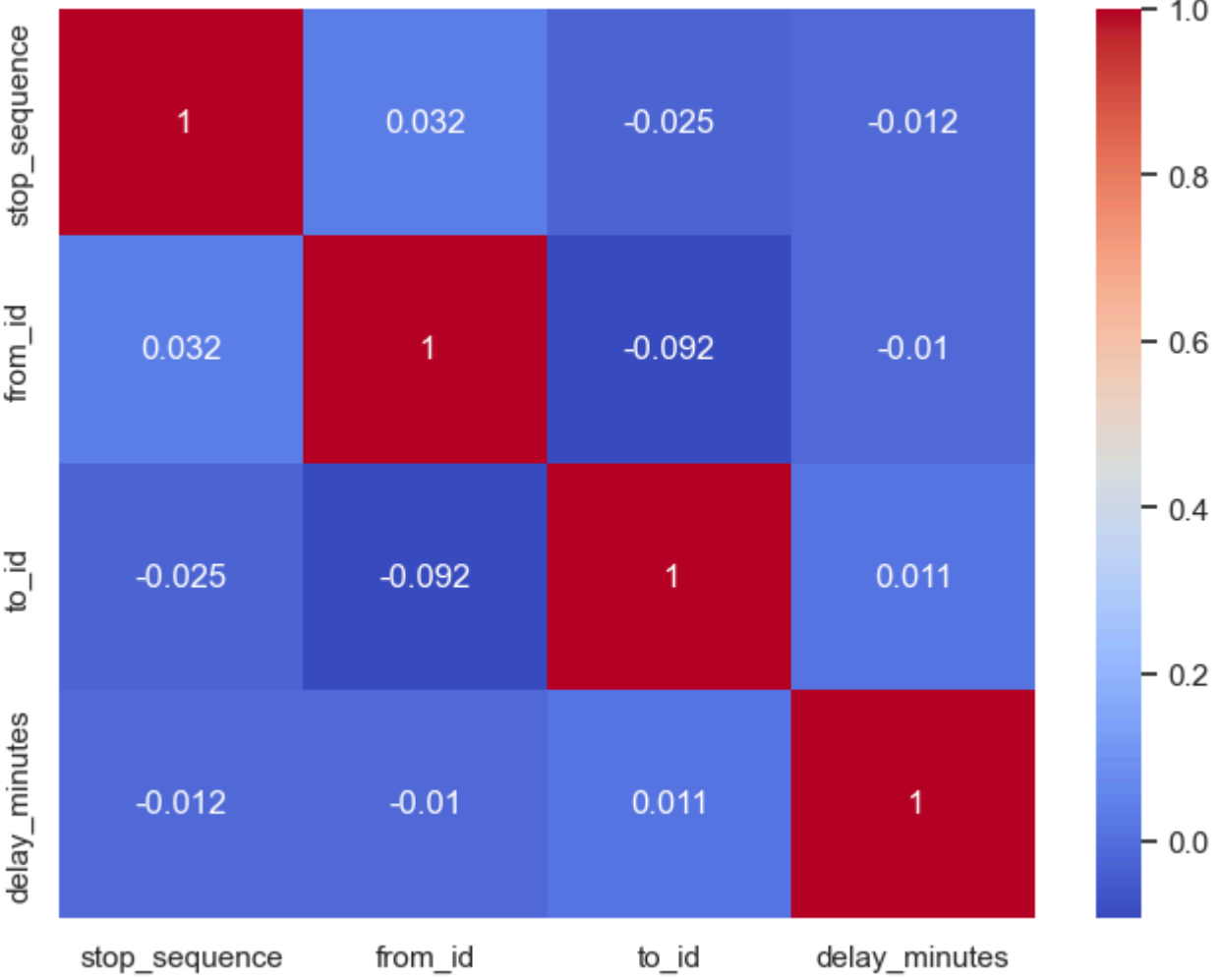
Extreme delay observations were identified using boxplot. These values were retained because they represent genuine disruption events rather than measurement errors.

```
1 #MISSING VALUE TREATMENT
2 OTP_df['delay_minutes'] = OTP_df['delay_minutes'].fillna(
3     OTP_df['delay_minutes'].median()
4 )
5
6 OTP_df['status'] = OTP_df['status'].fillna('Unknown')
7 OTP_df['line'] = OTP_df['line'].fillna('Unknown')
```

```
1 import numpy as np
2 #Correlation for numeric Variables
3 numeric = OTP_df.select_dtypes(
4     include=np.number
5 )
6
7 numeric.corr()
```

	stop_sequence	from_id	to_id	delay_minutes
stop_sequence	1.000000	0.031703	-0.025489	-0.011766
from_id	0.031703	1.000000	-0.092192	-0.010394
to_id	-0.025489	-0.092192	1.000000	0.010880
delay_minutes	-0.011766	-0.010394	0.010880	1.000000

```
1 #Correlation plot
2 plt.figure(figsize=(8,6))
3 sns.heatmap(
4     numeric.corr(),
5     annot=True,
6     cmap="coolwarm"
7 )
8
9 plt.show()
```



```
1 #Saving clean dataset
2 OTP_df.to_csv(
3     "combined_dataset.csv",
4     index=False
5 )
```

```
1 #Creating a copy of clean dataset for feature engineering
2 OTP_Copy=OTP_df.copy()
```

```
1 #Verifying Date and time columns
2 print(OTP_Copy[['date','scheduled_time','actual_time']].dtypes)
```

```
date datetime64[ns]
scheduled_time datetime64[ns]
actual_time datetime64[ns]
dtype: object
```

Hour feature captures the scheduled operating hour and allows the identification of peak congestion period.

```
1 #FEATURE ENGINEERING
2
3 #Extracting hour from the scheduled time
4 OTP_Copy['hour'] = OTP_Copy['scheduled_time'].dt.hour
5 OTP_Copy[['scheduled_time',"hour"]].tail()
```

	scheduled_time	hour
965563	2020-05-18 17:16:00	17.0
965564	2020-05-18 17:21:00	17.0
965565	2020-05-18 17:24:00	17.0
965566	2020-05-18 17:30:00	17.0
965567	2020-05-18 17:42:00	17.0

- Day of week: Different weekday can have different passenger demand.
- Month: Used for seasonal trend analysis
- Saturday=5, Sunday= 6
- rush_hour flag: 1 - rush hour & 0 - outside of rush hour. Rush hour correspond to peak commuter demand and are expected to have higher delay frequency. Morning Rush hour from 0700-1000 hours and Afternoon from 1600-1900 hours.
- Season will depends on the month that is: Dec, Jan , Feb: Winter March, April , May: Spring June, July , August: Summer Sept, Oct, Nov: Fall
- OTP_Binary : If delay is less than equal to 5 minutes than it will return 1 else 0. OTP Indicator was created using the industry threshold of five minutes.


```
1 #Similarly extracting Day of the week, month, weekend, Rush Hour, Season, OTP (1,0)
2 OTP_Copy['day_of_week'] = OTP_Copy['date'].dt.day_name()
3
4 OTP_Copy['month'] = OTP_Copy['date'].dt.month
5
6 OTP_Copy['weekend'] = np.where(OTP_Copy['date'].dt.dayofweek >=5,1,0)
7
8 OTP_Copy['rush_hour'] = np.where(
9     ((OTP_Copy['hour']>=7)&(OTP_Copy['hour']<=10)))
10     ((OTP_Copy['hour']>=16)&(OTP_Copy['hour']<=19))),
11     1,
12     0
13 )
14
15 def season(month):
16     if month in {12,1,2}:
17         return "Winter"
18     elif month in {3,4,5}:
19         return "Spring"
20     elif month in {6,7,8}:
21         return "Summer"
22     else:
23         return "Fall"
24
25 OTP_Copy["season"] = OTP_Copy["month"].apply(season)
26
27 OTP_Copy["otp_binary"] = np.where(
28     OTP_Copy["delay_minutes"]<=5,
29     1,
30     0
31 )
32
33 OTP_Copy[["from", "to", "date", "day_of_week", "month", "weekend", "rush_hour", "season", "otp_binary"]].head()
```

	from	to	date	day_of_week	month	weekend	rush_hour	season	otp_binary
0	Hoboken	Hoboken	2020-01-01	Wednesday	1	0	0	Winter	1
1	Hoboken	Newark Broad Street	2020-01-01	Wednesday	1	0	0	Winter	0
2	Newark Broad Street	Watessing Avenue	2020-01-01	Wednesday	1	0	0	Winter	1
3	Watessing Avenue	Bloomfield	2020-01-01	Wednesday	1	0	0	Winter	1
4	Bloomfield	Glen Ridge	2020-01-01	Wednesday	1	0	0	Winter	1

Average Station Delay Represents the histotical delay experience at each station.

```
1 #STATION RELIABILITY- Highest average delay most unreliable
2
3 station_delay = OTP_Copy.groupby("from")["delay_minutes"].mean()
4 OTP_Copy["avg_station_delay"] = ( OTP_Copy["from"].map(station_delay))
5 OTP_Copy[["from", "avg_station_delay"]].sort_values(by="avg_station_delay",
6 ascending=False).sample(10)
```

	from	avg_station_delay
825231	Little Silver	2.599929
498386	Netherwood	3.285863
856528	Brick Church	3.325271
99263	New Bridge Landing	2.957061
70417	Woodbridge	3.298058
50845	Newark Broad Street	3.960791
106079	Woodbridge	3.298058
484198	Red Bank	4.155992
496357	East Orange	2.893472
206699	Hillsdale	3.316559

ROUTE DELAY FREQUENCY

```
1 #Creating the delay frequency
2 route_frequency = (OTP_Copy.groupby("line")["delay_minutes"].count())
3
4 ##Merging with the dataset
5 OTP_Copy["route_delay_frequency"] = (OTP_Copy["line"].map(route_frequency))
6 OTP_Copy[["line", "route_delay_frequency"]].sort_values(by="route_delay_frequency",ascending=False).sample(10)
```

	line	route_delay_frequency
7807	No Jersey Coast	150754
142665	Pascack Valley	59708
68501	REGIONAL	10135
358207	Pascack Valley	59708
119606	Gladstone Branch	70615
372652	Bergen Co. Line	73221
285983	Northeast Corrdr	145825
348456	Pascack Valley	59708
474415	Raritan Valley	80413
605937	Bergen Co. Line	73221

```
1 #Historical Route reliability: How reliable this line historically is
2 route_reliability = (OTP_Copy.groupby("line")["otp_binary"].mean())
3 OTP_Copy["historical_route_reliability"] = (OTP_Copy["line"].map(route_reliability))
4
5 OTP_Copy[["line", "historical_route_reliability"]].sort_values(by="historical_route_reliability",
6 ascending=False).head()
```

	line	historical_route_reliability
752214	REGIONAL	1.0
553503	ACELA EXPRESS	1.0
30150	REGIONAL	1.0
30151	REGIONAL	1.0
30152	REGIONAL	1.0

```
1 #OPERATOR COMPARISON
2 OTP_Copy.groupby('type')[ 'delay_minutes'].agg([ 'mean', 'median', 'std'])
```

	mean	median	std
type			
Amtrak	2.083333	2.083333	0.000000
NJ Transit	3.662137	2.083333	8.284601

```
1 #Verifying new features
2
3 new_features = [
4
5     "hour",
6
7     "day_of_week",
8
9     "month",
10
11     "weekend",
12
13     "rush_hour",
14
15     "season",
16
17     "otp_binary",
18
19     "avg_station_delay",
20
21     "route_delay_frequency",
22
23     "historical_route_reliability"
24
25 ]
26
27 OTP_Copy[new_features].head()
```

	hour	day_of_week	month	weekend	rush_hour	season	otp_binary	avg_station_delay	route_delay_frequency	historical_route_reliability
0	20.0	Wednesday	1	0	0	Winter	1	2.812073	65923	0.755745
1	20.0	Wednesday	1	0	0	Winter	0	2.812073	65923	0.755745
2	20.0	Wednesday	1	0	0	Winter	1	3.960791	65923	0.755745
3	20.0	Wednesday	1	0	0	Winter	1	6.830793	65923	0.755745
4	20.0	Wednesday	1	0	0	Winter	1	5.327146	65923	0.755745

```
1 #Save feature engineered dataset
2 OTP_Copy.to_csv(
3     "feature_engineered_dataset.csv",
4     index=False
5 )
```

```
1 print("Dataset Shape:", OTP_Copy.shape)
2 print("\nColumns")
3 print(OTP_Copy.columns)
4 print("\nData Types")
5 print(OTP_Copy.dtypes)
```

Dataset Shape: (965568, 23)

Columns

Index(['date', 'train_id', 'stop_sequence', 'from', 'from_id', 'to', 'to_id', 'scheduled_time', 'actual_time', 'delay_minutes', 'status', 'line', 'type', 'hour', 'day_of_week', 'month', 'weekend', 'rush_hour', 'season', 'otp_binary', 'avg_station_delay', 'route_delay_frequency', 'historical_route_reliability'], dtypes='object')

Data Types

date	datetime64[ns]
train_id	object
stop_sequence	float64
from	object
from_id	float64
to	object
to_id	float64
scheduled_time	datetime64[ns]
actual_time	datetime64[ns]
delay_minutes	float64
status	object
line	object
type	object
hour	float64
day_of_week	object
month	int32
weekend	int64
rush_hour	int64
season	object
otp_binary	int64
avg_station_delay	float64
route_delay_frequency	int64
historical_route_reliability	float64
dtype:	object

=====

=====

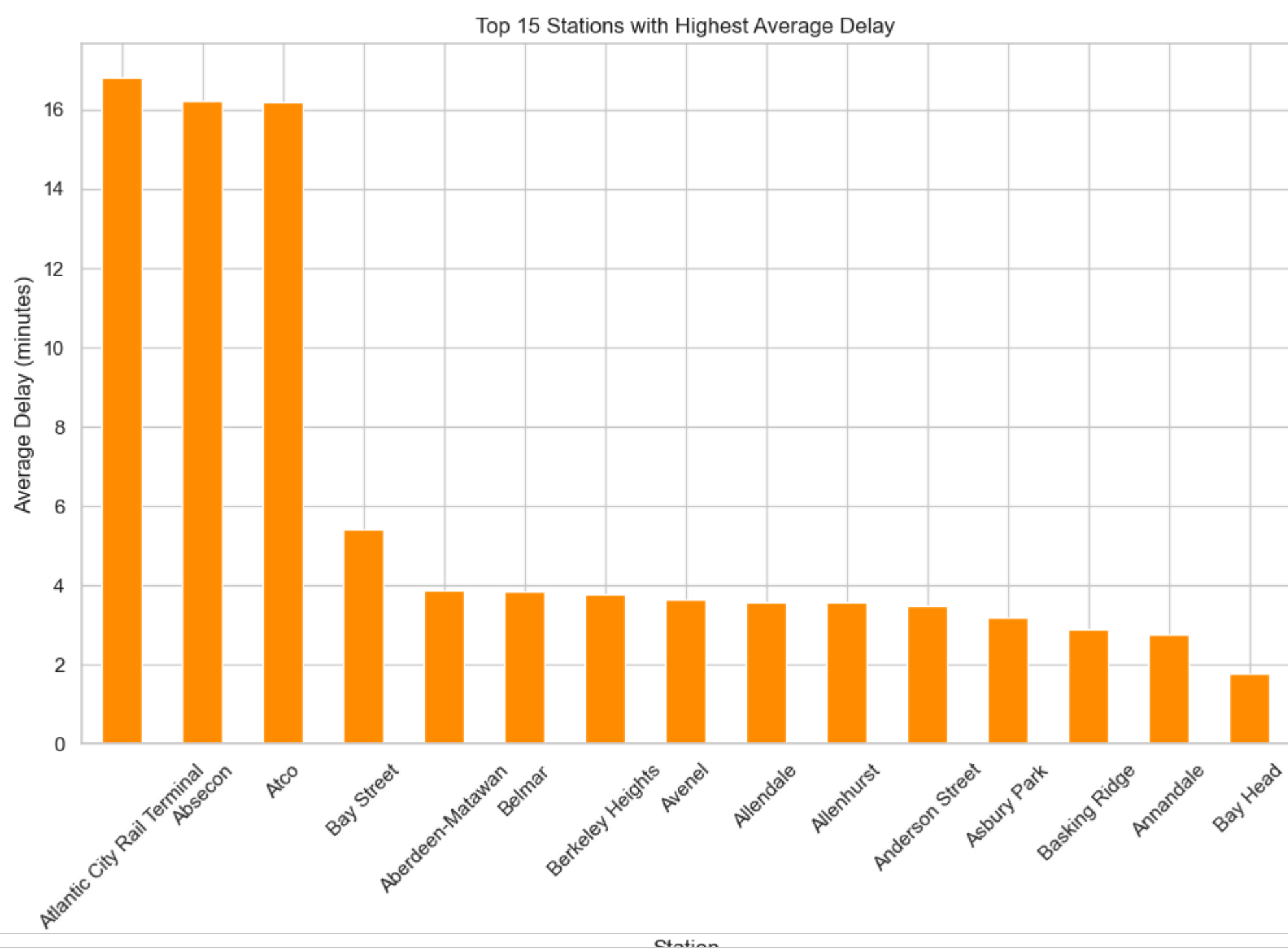
RESEARCH QUESTIONS

STATION RELIABILITY

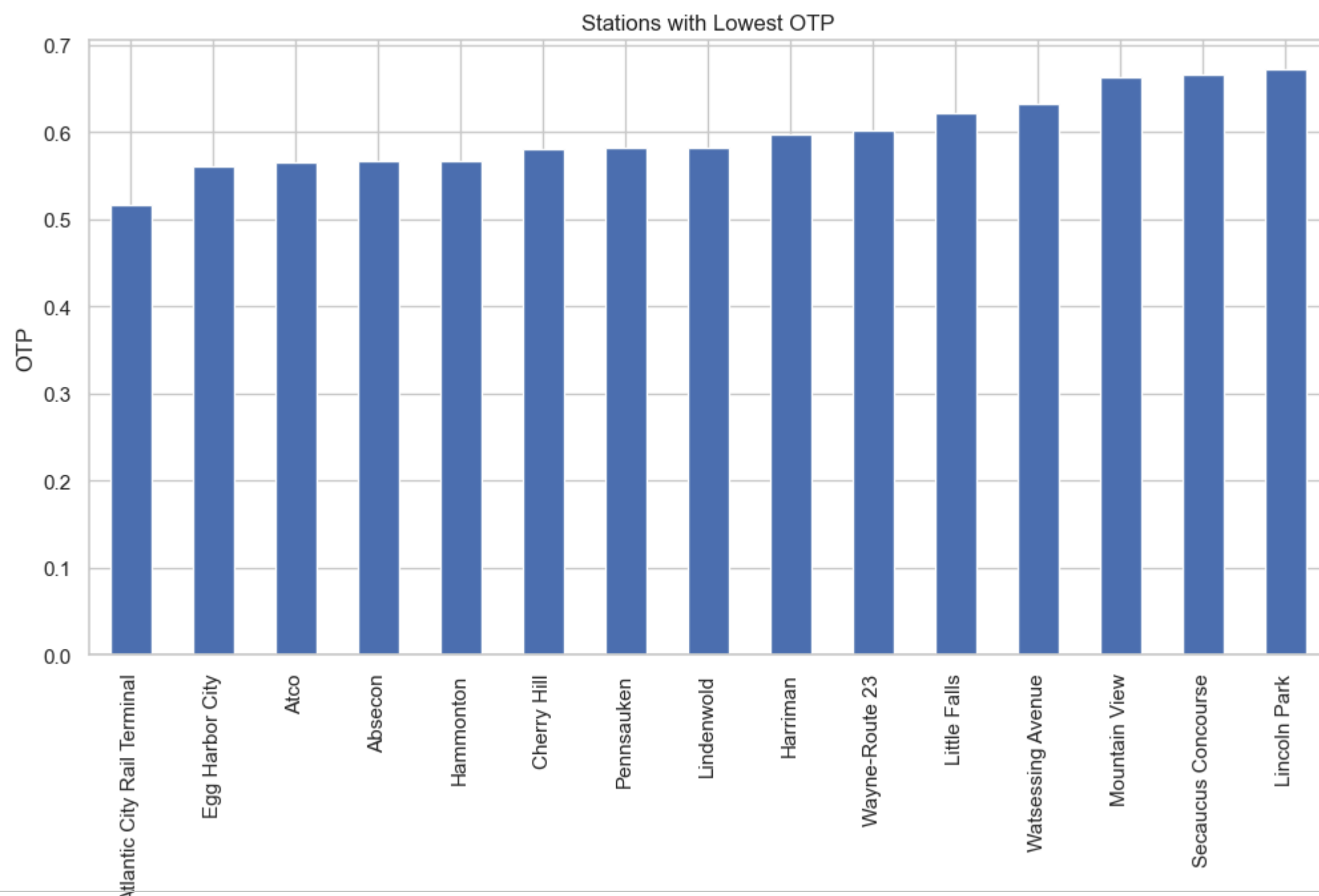
```
1 plt.figure(figsize=(12,7))
2
```



```
3 station_delay.head(15).sort_values(ascending=False).plot(
4     kind="bar",
5     color="darkorange"
6 )
7
8 plt.ylabel("Average Delay (minutes)")
9
10 plt.xlabel("Station")
11
12 plt.title("Top 15 Stations with Highest Average Delay")
13
14 plt.xticks(rotation=45)
15
16 plt.show()
```



```
1 station_otp = (OTP_Copy
2 .groupby("from")["otp_binary"].mean()).sort_values()
3 plt.figure(figsize=(12,6))
4 station_otp.head(15).sort_values(ascending=True).plot.bar()
5 plt.ylabel("OTP")
6 plt.title("Stations with Lowest OTP")
7 plt.show()
```



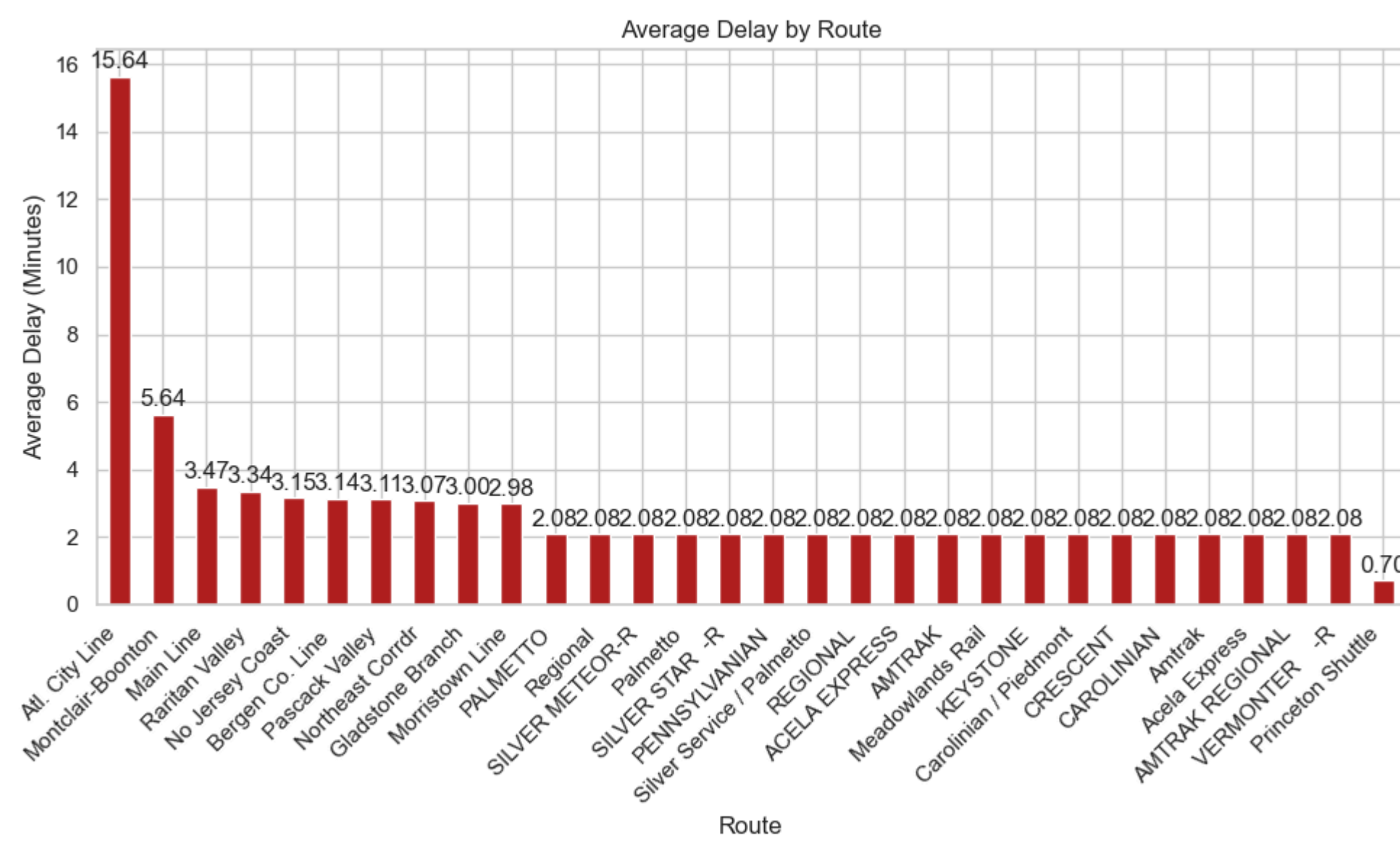
Station reliability is analyzed by calculating average delay minutes and on-time performance by station. Stations with higher average delays may indicate operational bottlenecks, congestion points, or infrastructure-related inefficiencies. Stations with lower OTP values represent locations where trains were less consistently on time

ROUTE RELIABILITY

```

1 route_delay = (
2     OTP_Copy
3     .groupby("line")["delay_minutes"]
4     .mean()
5     .sort_values(ascending=False)
6 )
7
8 plt.figure(figsize=(10, 6))
9
10 ax = route_delay.plot.bar(color="firebrick")
11
12 # Add data labels above each bar
13 ax.bar_label(
14     ax.containers[0],
15     fmt="%2f",          # Show two decimal places
16     padding=2
17 )
18
19 plt.xlabel("Route")
20 plt.ylabel("Average Delay (Minutes)")
21 plt.title("Average Delay by Route")
22 plt.xticks(rotation=45, ha="right")
23 plt.tight_layout()
24 plt.show()

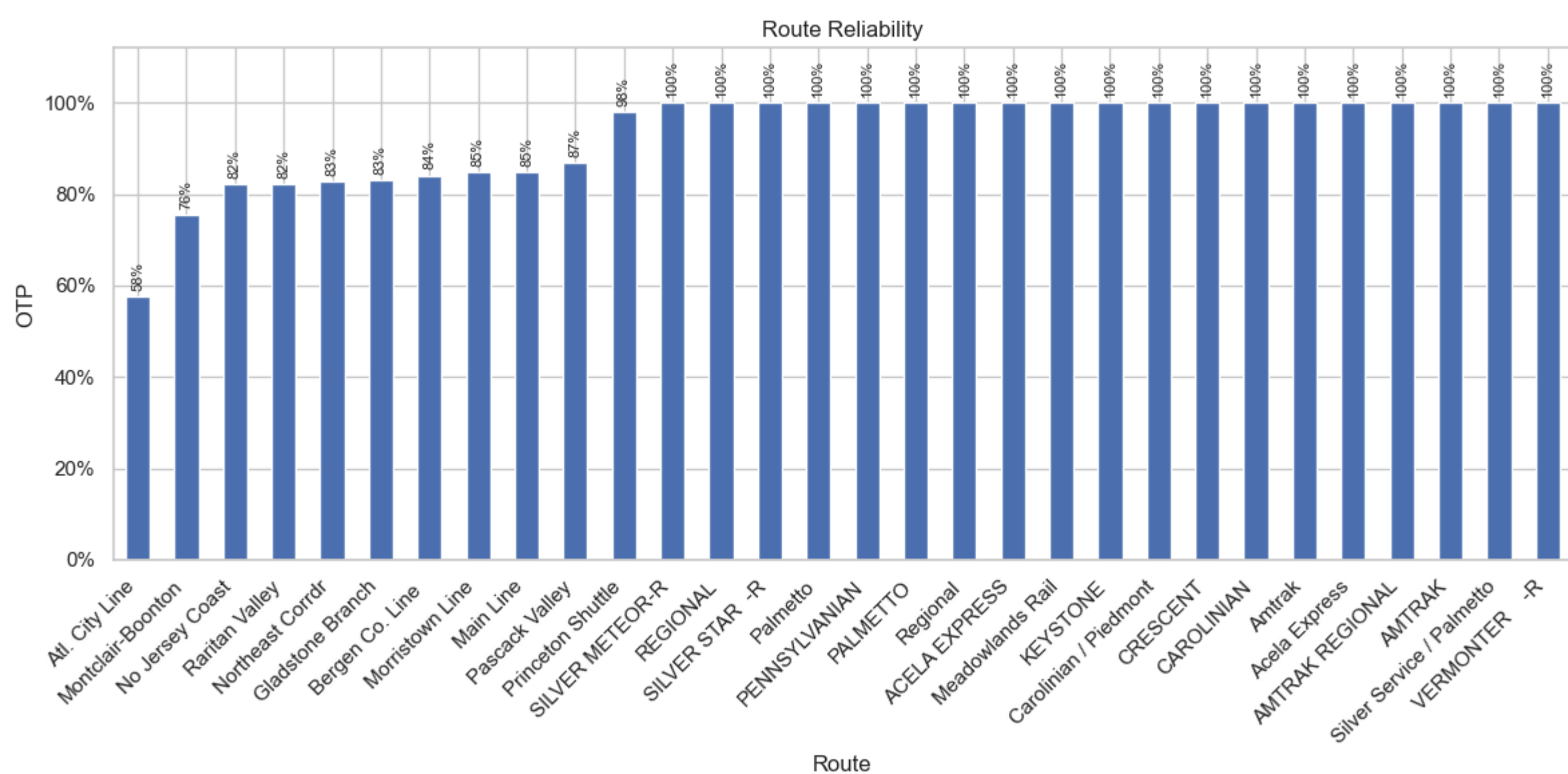
```



```

1 from matplotlib.ticker import PercentFormatter
2
3 # Route OTP
4 route_otp = (
5     OTP_Copy
6     .groupby("line")["otp_binary"]
7     .mean()
8     .sort_values()
9 )
10
11 plt.figure(figsize=(12, 6))
12
13 ax = route_otp.plot.bar()
14
15 # Add whole-number percentage labels
16 ax.bar_label(
17     ax.containers[0],
18     labels=f"({value * 100:.0f}%" for value in route_otp],
19     padding=3,
20     rotation=90,
21     fontsize=90,
22 )
23
24 # Format y-axis as whole-number percentages
25 ax.yaxis.set_major_formatter(PercentFormatter(xmax=1, decimals=0))
26
27 plt.xlabel("Route")
28 plt.ylabel("OTP")
29 plt.title("Route Reliability")
30 plt.ylim(0, 1.12)
31 plt.xticks(rotation=45, ha="right")
32 plt.tight_layout()
33 plt.show() my

```



7/25/26, 3:39 PM

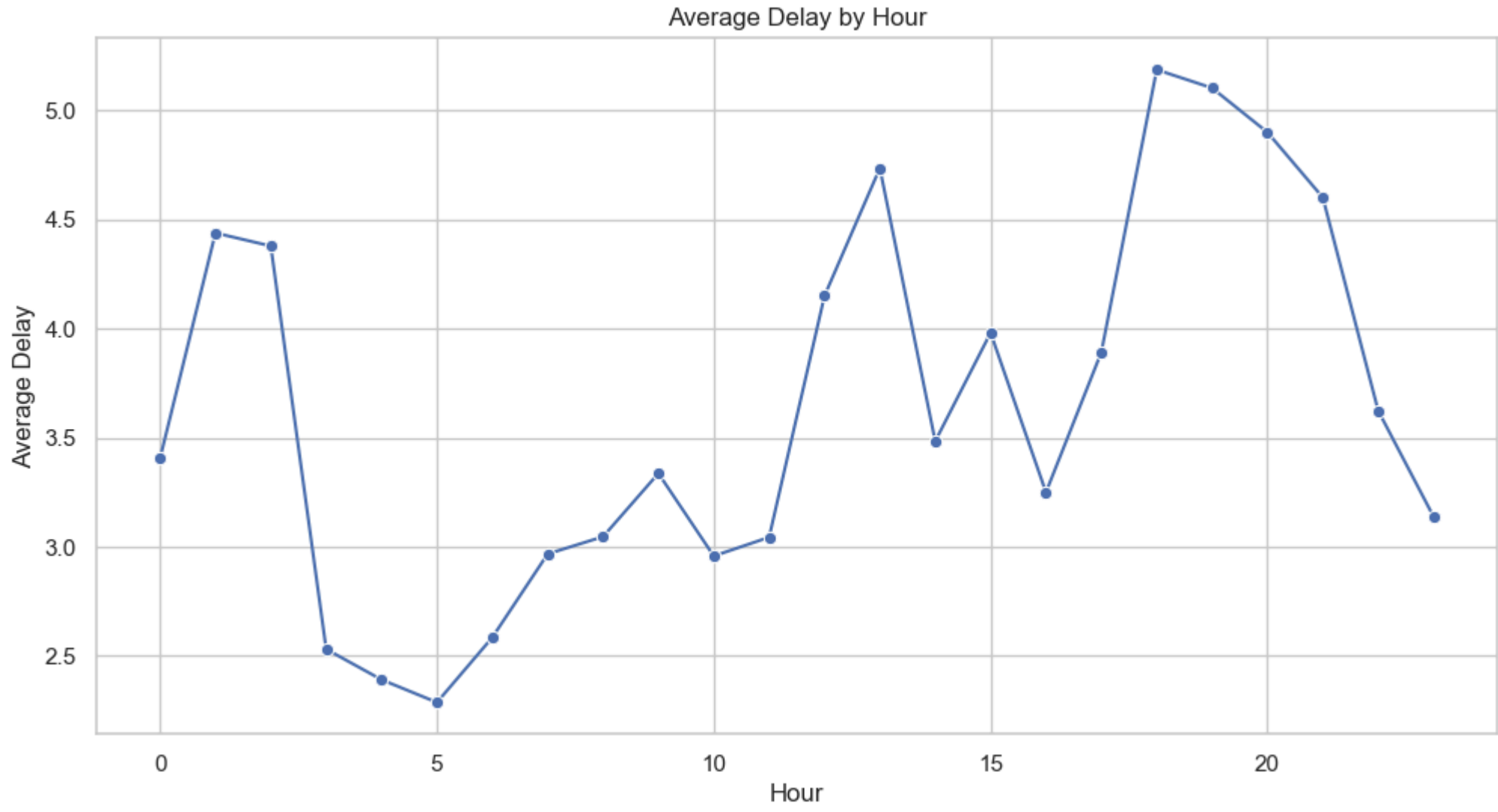
Route reliability is analyzed by grouping data by line. This helped compare average delay and OTP across different routes. Routes with higher average delay and lower OTP can be considered less reliable. This directly answers whether certain routes perform worse than others in terms of punctuality.

TEMPORAL DELAY PATTERNS Temporal delay patterns are analyzed using hour, day of week, weekend, month, and rush-hour features.

- The hourly delay trend helped identify whether delays increased during certain times of day.
- The rush-hour variable allowed comparison between peak and non-peak periods.
- Day-of-week analysis helped identify whether certain weekdays or weekends experienced higher delay levels.
- Monthly and seasonal analysis helped observe whether delay patterns changed from January through May.

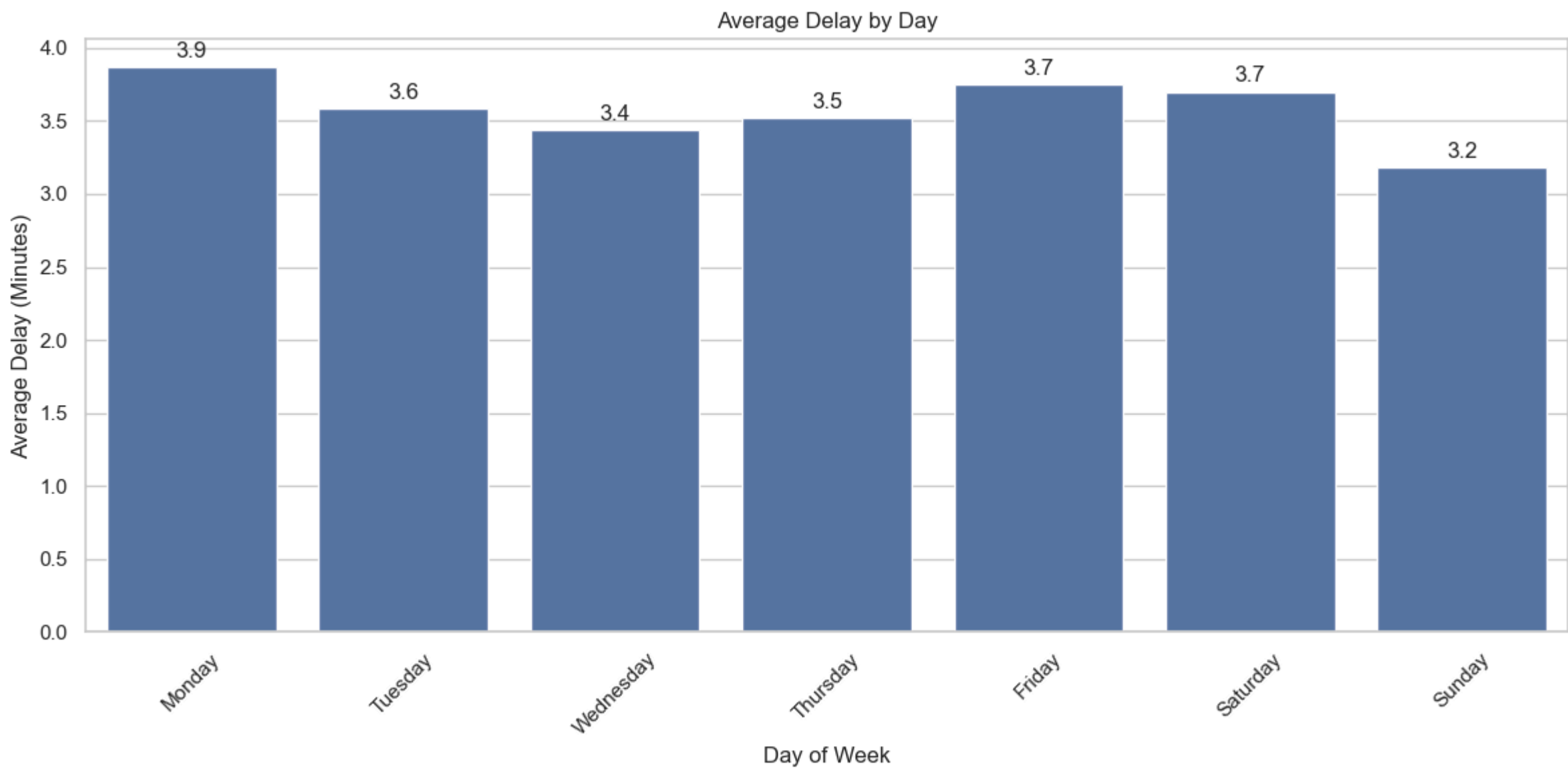
HOURLY DELAY PATTERN

```
1 hour_delay = (  
2 OTP_Copy  
3 .groupby("hour")  
4 ["delay_minutes"]  
5 .mean()  
6 )  
7 plt.figure(figsize=(12,6))  
8 sns.lineplot(  
9 x=hour_delay.index,  
10 y=hour_delay.values,  
11 marker="o"  
12 )  
13 plt.xlabel("Hour")  
14 plt.ylabel("Average Delay")  
15 plt.title("Average Delay by Hour")  
16 plt.show()
```



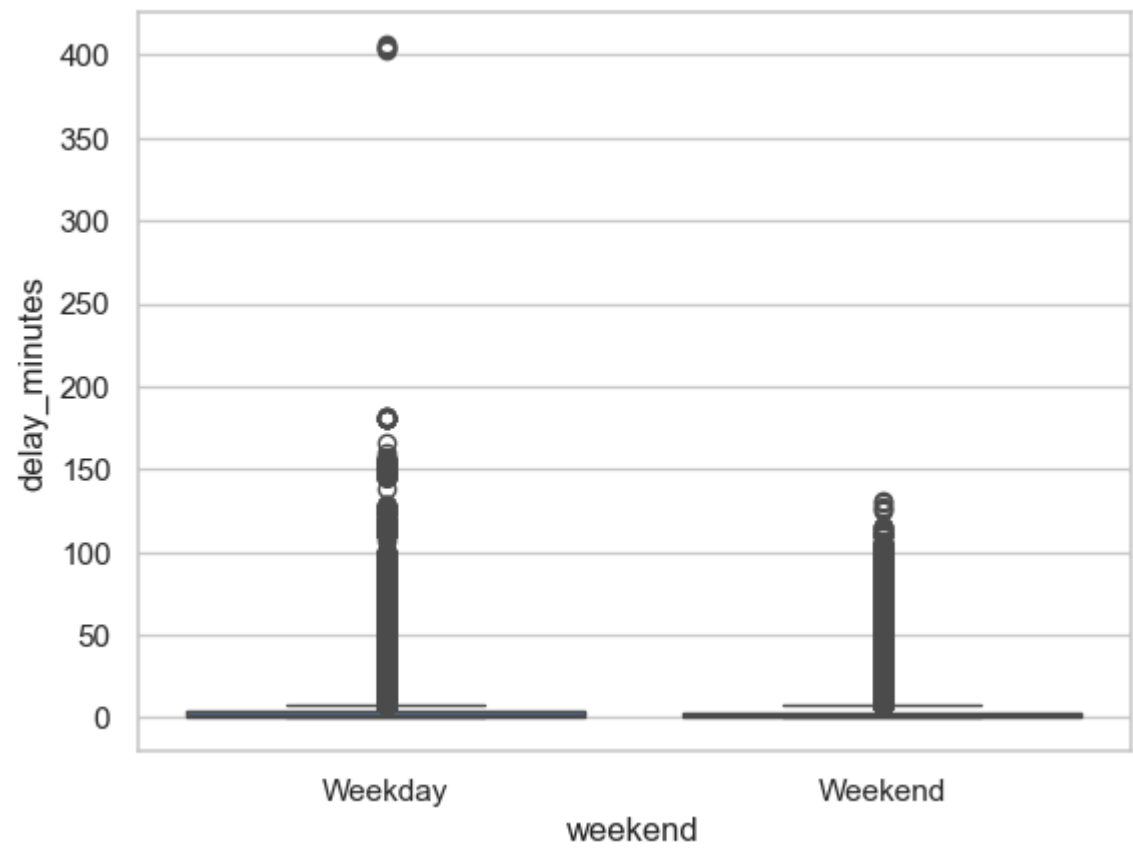
DAY OF WEEK

```
1 # Average delay by day  
2 day_delay = (  
3 OTP_Copy  
4 .groupby("day_of_week")["delay_minutes"]  
5 .mean()  
6 .reindex([  
7     "Monday",  
8     "Tuesday",  
9     "Wednesday",  
10    "Thursday",  
11    "Friday",  
12    "Saturday",  
13    "Sunday"  
14 ]) )  
15 )  
16  
17 plt.figure(figsize=(12, 6))  
18  
19 ax = sns.barplot(  
20 x=day_delay.index,  
21 y=day_delay.values  
22 )  
23  
24 # Add data labels above the bars  
25 ax.bar_label(  
26 ax.containers[0],  
27 fmt="%s.1f",  
28 padding=3  
29 )  
30  
31 plt.xticks(rotation=45)  
32 plt.xlabel("Day of Week")  
33 plt.ylabel("Average Delay (Minutes)")  
34 plt.title("Average Delay by Day")  
35 plt.tight_layout()  
36 plt.show()
```



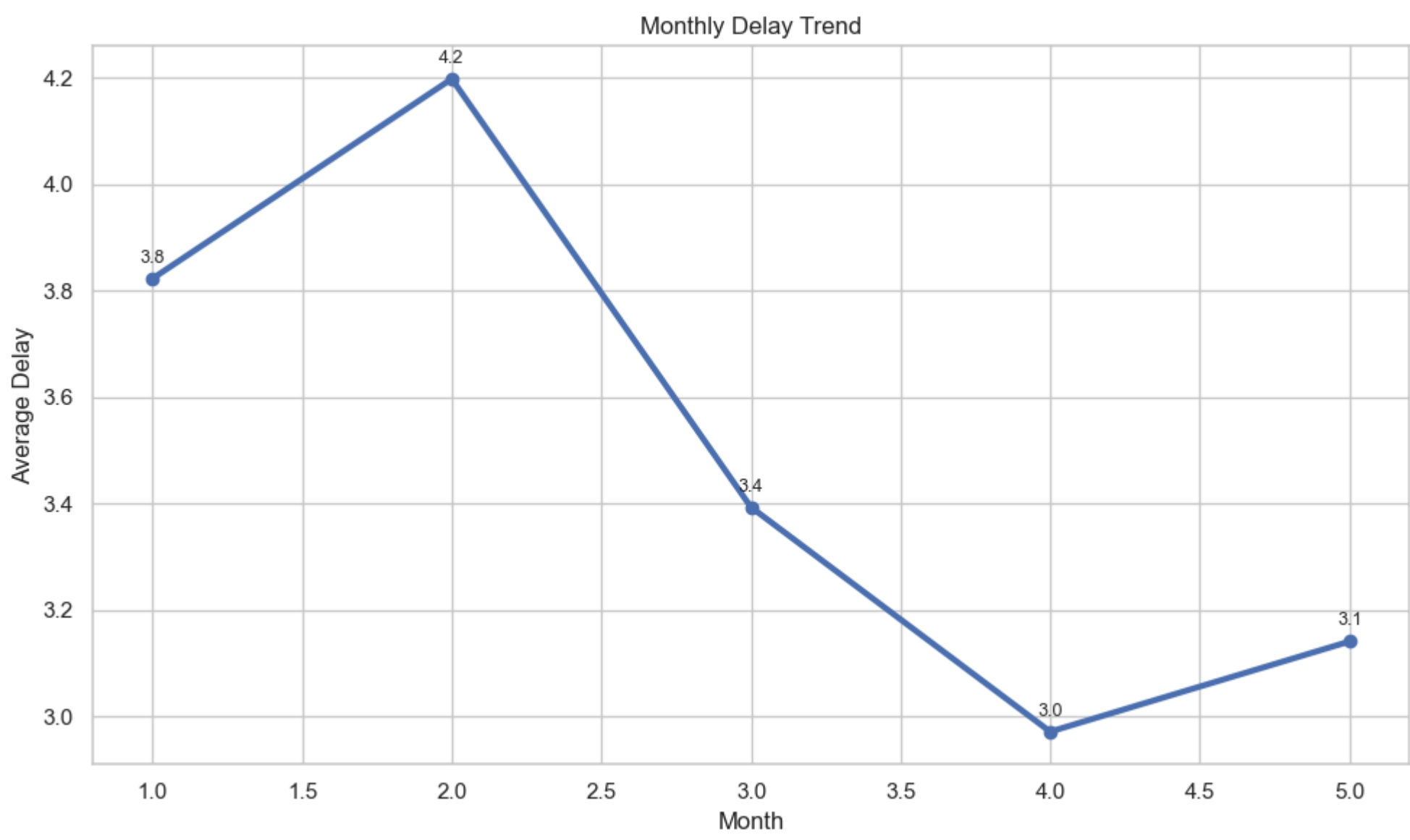
WEEKEND COMPARISON

```
1 sns.boxplot(  
2 data=OTP_Copy,  
3 x="weekend",  
4 y="delay_minutes"  
5 )  
6 plt.xticks(  
7 [0,1],  
8 ["Weekday","Weekend"]  
9 )  
10 plt.show()
```



MONTHLY TREND

```
1 month_delay = (  
2 OTP_Copy  
3 .groupby("month")["delay_minutes"]  
4 .mean()  
5 .sort_index()  
6 )  
7  
8 ax = month_delay.plot(  
9 marker="o",  
10 linewidth=3,  
11 figsize=(10, 6)  
12 )  
13  
14 # Add data labels to each point  
15 for month, value in month_delay.items():  
16     ax.annotate(  
17         f"{value:.1f}",  
18         xy=(month, value),  
19         xytext=(0, 8),  
20         textcoords="offset points",  
21         ha="center",  
22         fontsize=9  
23     )  
24  
25 plt.xlabel("Month")  
26 plt.ylabel("Average Delay")  
27 plt.title("Monthly Delay Trend")  
28 plt.tight_layout()  
29 plt.show()
```

=====

=====

RESEARCH QUESTION 2

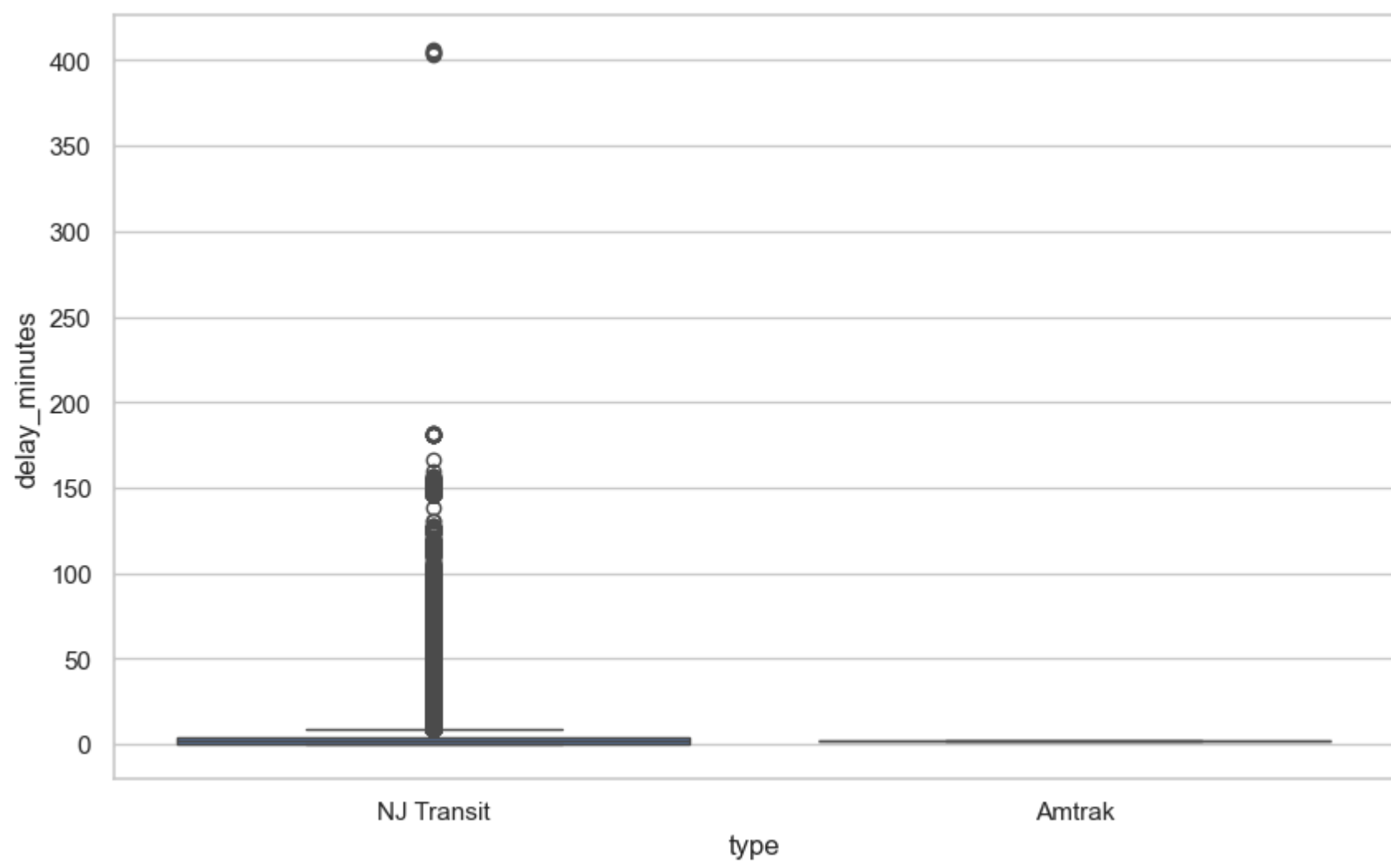
OPERATOR COMPARISON

The type column is used to compare delay behavior between NJ Transit and Amtrak. Grouping by operator allowed comparison of average delay, standard deviation, and record count.

- Boxplots were also used to compare delay spread between the two operators.

This analysis supports the second research question by showing whether there are measurable differences in delay behavior between the two operators on the same corridor.

```
1 operator_delay = (  
2 OTP_Copy  
3 .groupby("type")  
4 ["delay_minutes"]  
5 .agg(["mean", "std", "count"])  
6 )  
7 operator_delay  
8  
9 plt.figure(figsize=(10,6))  
10 sns.boxplot(  
11 data=OTP_Copy,  
12 x="type",  
13 y="delay_minutes"  
14 )  
15 plt.show()
```



=====

=====

RESEARCH QUESTION 3

Can machine learning predict future delays?

- A time-based train-test split was used instead of a random split.
- January to April data was used for training, while May data was used for testing. This is appropriate because the research question focuses on predicting future delays. Using earlier months for training and a later month for testing simulates a real-world forecasting scenario.

Three baseline models were used:

- Linear Regression was used as the simplest baseline model. It assumes a linear relationship between features and delay minutes.
- Support Vector Regression was used to test whether a more flexible model could capture non-linear delay patterns.
- Random Forest Regressor was used because it can capture complex relationships between route, station, operator type, and time-based features.

Interpretation The baseline machine learning models help determine whether historical train data has predictive value. If the models produce low MAE and RMSE values, it suggests that historical route, station, operator, and time-based patterns can support delay prediction.

If R^2 is low, it means delays are difficult to fully predict using the available features alone, likely because train delays are affected by external factors not included in the dataset, such as weather, mechanical failures, signal issues, crew availability, passenger incidents, or infrastructure disruptions.

Therefore, machine learning can be used as an initial decision-support tool for predicting future delay risk, but the prediction quality depends heavily on the available features. The current model provides a strong baseline and can be improved by adding more operational variables, weather data, incident reports, train volume, and historical delay lag features.

```
1 #MACHINE LEARNING FEATURES  
2  
3 # =====  
4 # 1. Import required libraries  
5 # =====  
6  
7 import numpy as np  
8 import pandas as pd  
9  
10 from sklearn.compose import ColumnTransformer  
11 from sklearn.pipeline import Pipeline  
12 from sklearn.preprocessing import OneHotEncoder, StandardScaler  
13 from sklearn.impute import SimpleImputer  
14  
15 from sklearn.linear_model import LinearRegression  
16 from sklearn.ensemble import RandomForestRegressor  
17  
18 from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score  
19  
20  
21 # =====  
22 # 2. Sort data by date  
23 # =====  
24 # Since we are predicting future delays, the dataset should be sorted by date.  
25 # This makes sure older data is used for training and future data is used for testing.  
26  
27 df = OTP_Copy.sort_values("date").copy()  
28  
29  
30 # =====  
31 # 3. Define ML features and target  
32 # =====  
33 # These are the input variables used to predict delay_minutes.  
34 # Station-level variables were excluded from some model runs  
35 # because one-hot encoding created a high-dimensional feature space, making tree-based models computationally expensive.  
36  
37 features = [  
38     #"from",  
39     #"to",  
40     "line",  
41     "type",  
42     "status",  
43     "hour",  
44     "day_of_week",  
45     "month",  
46     "weekend",  
47     "rush_hour"  
48 ]  
49  
50 target = "delay_minutes"  
51  
52  
53 # =====  
54 # 4. Time-based train-test split  
55 # =====  
56 # January to April data is used for training.  
57 # May data is used for testing.  
58 # This supports the research question: can historical data predict future delays?  
59  
60  
61 train = df[df["month"] < 5]  
62 test = df[df["month"] >= 5]  
63  
64 X_train = train[features]  
65 y_train = train[target]  
66  
67 X_test = test[features]  
68 y_test = test[target]  
69  
70 # Remove rows where target is missing  
71 train_valid = y_train.notna()  
72 test_valid = y_test.notna()  
73  
74 X_train = X_train[train_valid]  
75 y_train = y_train[train_valid]  
76  
77 X_test = X_test[test_valid]  
78 y_test = y_test[test_valid]  
79  
80 # =====  
81 # 5. Identify numeric and categorical columns  
82 # =====  
83  
84 numeric_features = X_train.select_dtypes(include=["int64", "float64"]).columns  
85 categorical_features = X_train.select_dtypes(include=["object", "category", "bool"]).columns  
86  
87  
88 # =====  
89 # 6. Preprocessing for numeric variables  
90 # =====  
91 # Missing numeric values are filled with median.  
92 # StandardScaler is used because Linear Regression and SVR are sensitive to scale.  
93  
94 numeric_transformer = Pipeline(  
95     steps=[  
96         ("imputer", SimpleImputer(strategy="median")),  
97         ("scaler", StandardScaler())  
98     ]  
99 )
```



```
100
101
102 # =====
103 # 7. Preprocessing for categorical variables
104 # =====
105 # Missing categorical values are filled with the most frequent value.
106 # OneHotEncoder converts text categories into numeric format for ML models.
107 # handle_unknown="ignore" prevents errors if May has categories not seen in Jan-Apr.
108
109 categorical_transformer = Pipeline(
110     steps=[
111         ("imputer", SimpleImputer(strategy="most_frequent")),
112         ("encoder", OneHotEncoder(handle_unknown="ignore"))
113     ]
114 )
115
116 # =====
117 # 8. Combine preprocessing steps
118 # =====
119 preprocessor = ColumnTransformer(
120     transformers=[
121         ("num", numeric_transformer, numeric_features),
122         ("cat", categorical_transformer, categorical_features)
123     ]
124 )
125
126 # =====
127 # 9. Create models
128 # =====
129
130 models = {
131     "Linear Regression": LinearRegression(),
132     "Random Forest": RandomForestRegressor(
133         n_estimators=50,
134         random_state=42,
135         n_jobs=-1
136     )
137 }
138
139 # =====
140 # 10. Train and evaluate models
141 # =====
142
143 results = []
144
145 for model_name, model in models.items():
146
147     print(f"Training {model_name}...")
148
149     pipeline = Pipeline(
150         steps=[
151             ("preprocessor", preprocessor),
152             ("model", model)
153         ]
154     )
155
156     pipeline.fit(X_train, y_train)
157
158     print(f"Predicting with {model_name}...")
159
160     predictions = pipeline.predict(X_test)
161
162     mae = mean_absolute_error(y_test, predictions)
163     rmse = np.sqrt(mean_squared_error(y_test, predictions))
164     r2 = r2_score(y_test, predictions)
165
166     results.append({
167         "Model": model_name,
168         "MAE": mae,
169         "RMSE": rmse,
170         "R2 Score": r2
171     })
172
173 # =====
174 # 11. Display model comparison
175 # =====
176
177 results_df = pd.DataFrame(results)
178 results_df
```

Training Linear Regression...			
Predicting with Linear Regression...			
Training Random Forest...			
Predicting with Random Forest...			
	Model	MAE	RMSE R2 Score
0	Linear Regression	3.205252	9.171396 0.095508
1	Random Forest	2.806961	7.895864 0.329601

BASELINE MODEL The Linear Regression model: baseline regression model to predict the number of delay minutes. In the report, this model serves as a benchmark that can be compared against more advanced models like Gradient Boosting, or Random Forest.

SVR was initially considered as a regression model; however, due to the high number of categorical features and one-hot encoded station variables, the model was computationally expensive and did not complete efficiently. Therefore, faster and more interpretable models such as Linear Regression and Random Forest were prioritized for baseline and comparative evaluation.

EVALUATION METRICS Tells how well the model predicts delay minutes.

- Mean Absolute Error(MAE): The lower the better
- Root Mean Squared Error (RMSE) : The lower the better. Root Mean Squared Error, or RMSE, also measures error in minutes, but it penalizes larger errors more strongly. This is useful because extreme train delays are operationally important.
- R Square Score: Measures how much of the variation in delay is explained by the model. A value closer to 1 is better, while a value near 0 means the model does not explain much variation.

LINEAR REGRESSION:

MAE: The model's predictions differed from the actual delay by an average of approximately 3.20 minutes.

RMSE: Larger prediction errors average about 9.17 minutes.

R Square Score: The model explains approximately 10% of the variation in train delays.

It explained approximately 10% of the variation in train delay duration. An average prediction error of approximately 3 minutes indicates reasonable predictive capability using operational and temporal features alone.

RANDOM FOREST:

MAE: The model's predictions differed from the actual delay by an average of approximately 2.8 minutes.

RMSE: Larger prediction errors average about 7.89 minutes.

R Square Score: The model explains approximately 33% of the variation in train delays.

It explained approximately 33% of the variation in train delay duration. An average prediction error of approximately 2.8 minutes indicates reasonable predictive capability using operational and temporal features alone.

```
1 #GRADIENT BOOST FOR DELAY MINUTE PREDICTION
2 from sklearn.ensemble import GradientBoostingRegressor
3
4 # =====
5 # Gradient Boosting Model
6 # =====
7
8 gradient_boost_model = Pipeline(
9     steps=[
10         ("preprocessor", preprocessor),
11         ("model", GradientBoostingRegressor(
12             n_estimators=50,
13             learning_rate=0.05,
14             max_depth=3,
15             random_state=42
16         ))
17     ]
18 )
19
20 # Train the model
21 print("Training Gradient Boosting model...")
22 gradient_boost_model.fit(X_train, y_train)
23
24 # Make predictions
25 print("Making predictions...")
26 gb_predictions = gradient_boost_model.predict(X_test)
27
28 # Evaluate model
29 gb_mae = mean_absolute_error(y_test, gb_predictions)
30 gb_rmse = np.sqrt(mean_squared_error(y_test, gb_predictions))
31 gb_r2 = r2_score(y_test, gb_predictions)
32
33 print("Gradient Boosting Results")
34 print("MAE:", gb_mae)
35 print("RMSE:", gb_rmse)
36 print("R² Score:", gb_r2)
```

Training Gradient Boosting model...
Making predictions...
Gradient Boosting Results
MAE: 2.955845527108788
RMSE: 8.478995151842537
R² Score: 0.228381821447115

GRADIENT BOOST:

MAE: The model's predictions differed from the actual delay by an average of approximately 2.9 minutes.

RMSE: Larger prediction errors average about 8.47 minutes.

R Square Score: The model explains approximately 22% of the variation in train delays.

The Gradient Boosting model achieved an average prediction error of approximately 3 minutes, indicating reasonable predictive accuracy for routine train operations. However, the larger RMSE demonstrates that the model struggles to accurately predict extreme delay events, which substantially increase prediction errors. Furthermore, the R² score of 22.8% indicates that the model explains only a modest proportion of the variability in train delays. This suggests that while temporal and operational variables such as line, train type, status, hour, day of week, and rush-hour indicators contribute to predicting delays, a significant amount of variation is driven by external operational factors that were not available in the dataset.

```
1 import time
2 import pickle
3 import pandas as pd
4
5 from sklearn.base import clone
```

```
1 # =====
2 # REGRESSION COMPUTATIONAL COST
3 # =====
4
5 regression_compute_results = []
6
7 regression_timing_models = {
8     "Linear Regression": Pipeline(
9         steps=[
10             ("preprocessor", clone(preprocessor)),
11             ("model", LinearRegression())
12         ]
13     ),
14
15     "Random Forest Regressor": Pipeline(
16         steps=[
17             ("preprocessor", clone(preprocessor)),
18             (
19                 "model",
20                 RandomForestRegressor(
21                     n_estimators=50,
22                     random_state=42,
23                     n_jobs=-1
24                 )
25             )
26         ]
27     ),
28
29     "Gradient Boosting Regressor": clone(
30         gradient_boost_model
31     )
32 }
```



```
33
34 for model_name, timing_pipeline in regression_timing_models.items():
35
36     print(f"Measuring {model_name}...")
37
38     # Measure model training time
39     training_start = time.perf_counter()
40
41     timing_pipeline.fit(
42         X_train,
43         y_train
44     )
45
46     training_end = time.perf_counter()
47
48     training_time = (
49         training_end - training_start
50     )
51
52     # Measure prediction time
53     prediction_start = time.perf_counter()
54
55     timing_predictions = timing_pipeline.predict(
56         X_test
57     )
58
59     prediction_end = time.perf_counter()
60
61     prediction_time = (
62         prediction_end - prediction_start
63     )
64
65     # Number of predictions generated per second
66     predictions_per_second = (
67         len(X_test) / prediction_time
68         if prediction_time > 0
69         else 0
70     )
71
72     # Approximate fitted pipeline size in memory
73     model_size_mb = (
74         len(pickle.dumps(timing_pipeline))
75         / (1024 ** 2)
76     )
77
78     regression_compute_results.append(
79         {
80             "Task": "Regression",
81             "Model": model_name,
82             "Training Rows": len(X_train),
83             "Testing Rows": len(X_test),
84             "Training Time (Seconds)": training_time,
85             "Prediction Time (Seconds)": prediction_time,
86             "Predictions Per Second": predictions_per_second,
87             "Model Size (MB)": model_size_mb
88         }
89     )
90
91 regression_compute_table = pd.DataFrame(
92     regression_compute_results
93 )
94
95 regression_compute_table = regression_compute_table.round(
96     {
97         "Training Time (Seconds)": 4,
98         "Prediction Time (Seconds)": 4,
99         "Predictions Per Second": 2,
100        "Model Size (MB)": 4
101    }
102 )
103
104 print("\nRegression Computational Cost")
105 display(regression_compute_table)
```

Measuring Linear Regression...
Measuring Random Forest Regressor...
Measuring Gradient Boosting Regressor...

Regression Computational Cost

	Task	Model	Training Rows	Testing Rows	Training Time (Seconds)	Prediction Time (Seconds)	Predictions Per Second	Model Size (MB)
0	Regression	Linear Regression	866870	98698	3.4844	0.1723	572696.03	0.0036
1	Regression	Random Forest Regressor	866870	98698	81.3852	0.2246	439457.70	23.3664
2	Regression	Gradient Boosting Regressor	866870	98698	23.5403	0.2517	392196.53	0.0739

DELAYED VS ON TIME CLASSIFICATION

```
1 # Create clean binary target
2 #OTP_Copy["otp_binary"] = (OTP_Copy["delay_minutes"] > 0).astype(int)
3
4 # Check result
5 print(OTP_Copy["otp_binary"].value_counts())
```

otp_binary
1 803231
0 162337
Name: count, dtype: int64

```
1 # Sort by date
2 df = OTP_Copy.sort_values("date").copy()
3
4 features_Class = [
5     "from",
6     "to",
7     "line",
8     "type",
9     "status",
10    "hour",
11    "day_of_week",
12    "month",
13    "weekend",
14    "rush_hour"
15 ]
16
17 target_Class = "otp_binary"
18
19 # Time-based split
20 train = df[df["month"] < 5]
21 test = df[df["month"] >= 5]
22
23 X_train = train[features_Class]
24 y_train = train[target_Class]
25
26 X_test = test[features_Class]
27 y_test = test[target_Class]
```

```
1 #LOGISTIC REGRESSION
2 from sklearn.linear_model import LogisticRegression
3 from sklearn.metrics import (accuracy_score,precision_score,recall_score,f1_score)
4
5 logistic_model = Pipeline(
6     steps=[
7         ("preprocessor", preprocessor),
8         ("model", LogisticRegression(max_iter=1000))
9     ]
10 )
11
12 logistic_model.fit(X_train, y_train)
13
14 log_predictions = logistic_model.predict(X_test)
15
16 print("Logistic Accuracy :", accuracy_score(y_test, log_predictions))
17 print("Logistic Precision:", precision_score(y_test, log_predictions))
18 print("Logistic Recall :", recall_score(y_test, log_predictions))
19 print("Logistic F1 Score :", f1_score(y_test, log_predictions))
```

Logistic Accuracy : 0.8940780018399562
Logistic Precision: 0.89531803181355654
Logistic Recall : 0.8979896135987345
Logistic F1 Score : 0.9438298832542672

LOGISTIC REGRESSION Accuracy = 89.40% means the percentage of trains that were correctly classified as either on time or delayed. The Logistic Regression model correctly classified approximately 89 out of every 100 train observations.

Precision = Approximately 90% of the trains predicted as delayed were actually delayed. This means the model produces relatively few false alarms.

Recall = The model successfully identified over 99% of all delayed trains. This is particularly important in railway operations because failing to identify delays could negatively affect planning and operational response.

F1 Score = A score of 94.3% indicates that the model achieves an excellent balance between correctly identifying delayed trains while minimizing false delay predictions.

```
1 #RANDOM FOREST CLASSIFIER
2 from sklearn.ensemble import RandomForestClassifier
3
4 rf_model = Pipeline(
5     steps=[
6         ("preprocessor", preprocessor),
7         ("model", RandomForestClassifier(
8             n_estimators=100,
9             random_state=42,
10            n_jobs=-1
11        ))
12     ]
13 )
14
15 rf_model.fit(X_train, y_train)
16
17 rf_predictions = rf_model.predict(X_test)
18 print("Random Forest Accuracy :", accuracy_score(y_test, rf_predictions))
19 print("Random Forest Precision:", precision_score(y_test, rf_predictions))
20 print("Random Forest Recall :", recall_score(y_test, rf_predictions))
21 print("Random Forest F1 Score :", f1_score(y_test, rf_predictions))
```

Random Forest Accuracy : 0.8980526454436767
Random Forest Precision: 0.9857373636439338
Random Forest Recall : 0.98857183080584173
Random Forest F1 Score : 0.9453431399176587

RANDOM FOREST Random Forest performed similar to Logistic Regression across nearly every metric.

- Highest Accuracy (Logistic: 0.8940, Random Forest: 0.8980)
- Highest Recall (Logistic: 0.9979, Random Forest: 0.9885)
- Highest F1 Score (Logistic: 0.9438, Random Forest: 0.9453) The Recall of 98.85% means the model detected almost every delayed train. Operationally, this is valuable because very few delayed trains would be missed.

```
1 #GRADIENT BOOSTING CLASSIFIER
2 from sklearn.ensemble import GradientBoostingClassifier
3
4 gb_model = Pipeline(
5     steps=[
6         ("preprocessor", preprocessor),
7         ("model", GradientBoostingClassifier(
8             n_estimators=50,
9             learning_rate=0.05,
10            max_depth=3,
11            random_state=42
12        ))
13     ]
14 )
15
16 gb_model.fit(X_train, y_train)
17
18 gb_predictions = gb_model.predict(X_test)
19
20 print("Gradient Boost Accuracy :", accuracy_score(y_test, gb_predictions))
21 print("Gradient Boost Precision:", precision_score(y_test, gb_predictions))
22 print("Gradient Boost Recall :", recall_score(y_test, gb_predictions))
23 print("Gradient Boost F1 Score :", f1_score(y_test, gb_predictions))
```

Gradient Boost Accuracy : 0.8996433565016515
Gradient Boost Precision: 0.98076852830421
Gradient Boost Recall : 0.9973415736974848
Gradient Boost F1 Score : 0.946598303869399

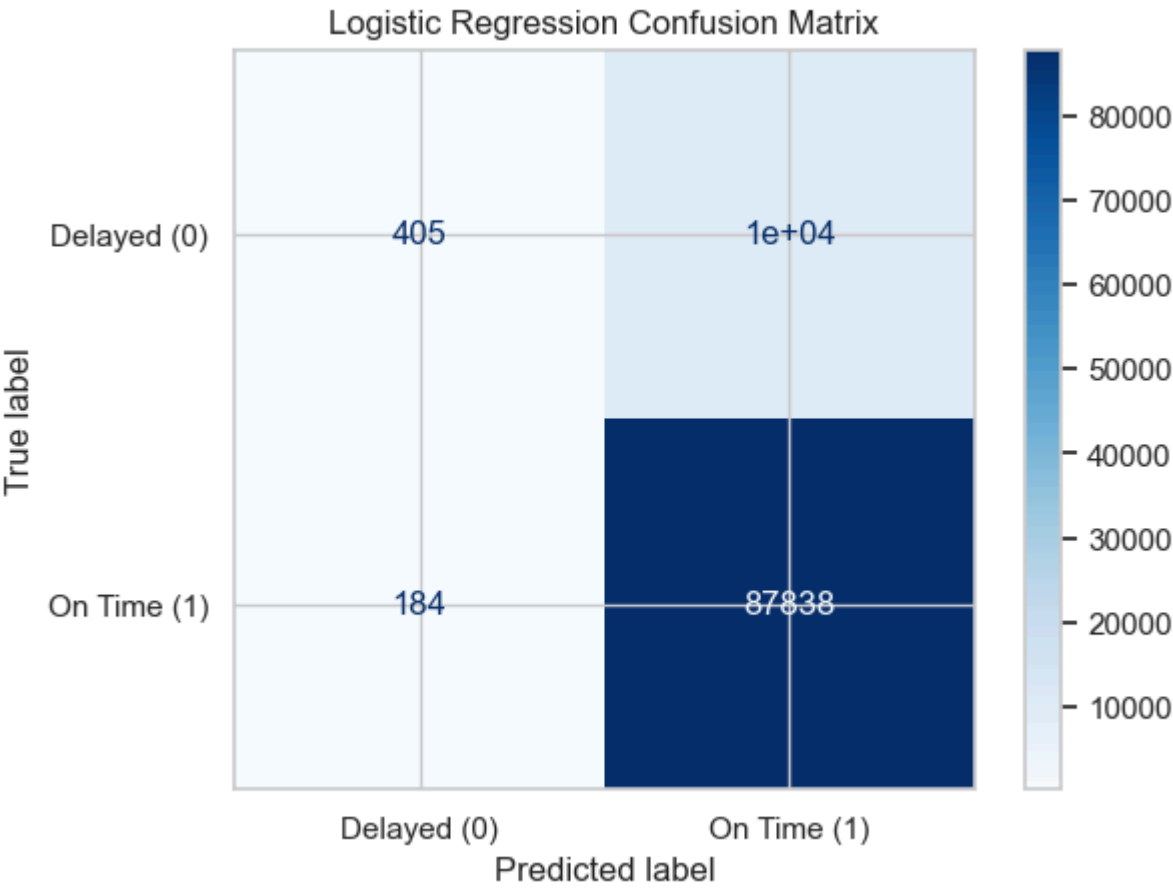

```
1 # =====
2 # CLASSIFICATION COMPUTATIONAL COST
3 # =====
4
5 classification_compute_results = []
6
7 classification_timing_models = {
8     "Logistic Regression": clone(logistic_model),
9     "Random Forest Classifier": clone(rf_model),
10    "Gradient Boosting Classifier": clone(gb_model)
11 }
12
13 for model_name, timing_pipeline in classification_timing_models.items():
14
15     print(f"Measuring {model_name}...")
16
17     # Measure model training time
18     training_start = time.perf_counter()
19
20     timing_pipeline.fit(
21         X_train,
22         y_train
23     )
24
25     training_end = time.perf_counter()
26
27     training_time = (
28         training_end - training_start
29     )
30
31     # Measure prediction time
32     prediction_start = time.perf_counter()
33
34     timing_predictions = timing_pipeline.predict(
35         X_test
36     )
37
38     prediction_end = time.perf_counter()
39
40     prediction_time = (
41         prediction_end - prediction_start
42     )
43
44     # Number of predictions generated per second
45     predictions_per_second = (
46         len(X_test) / prediction_time
47         if prediction_time > 0
48         else 0
49     )
50
51     # Approximate fitted pipeline size
52     model_size_mb = (
53         len(pickle.dumps(timing_pipeline))
54         / (1024 * 2)
55     )
56
57     classification_compute_results.append(
58         {
59             "Task": "Classification",
60             "Model": model_name,
61             "Training Rows": len(X_train),
62             "Testing Rows": len(X_test),
63             "Training Time (Seconds)": training_time,
64             "Prediction Time (Seconds)": prediction_time,
65             "Predictions Per Second": predictions_per_second,
66             "Model Size (MB)": model_size_mb
67         }
68     )
69
70 classification_compute_table = pd.DataFrame(
71     classification_compute_results
72 )
73
74 classification_compute_table = (
75     classification_compute_table.round(
76         {
77             "Training Time (Seconds)": 4,
78             "Prediction Time (Seconds)": 4,
79             "Predictions Per Second": 2,
80             "Model Size (MB)": 4
81         }
82     )
83 )
84
85 print("\nClassification Computational Cost")
86 display(classification_compute_table)
```

Measuring Logistic Regression...
Measuring Random Forest Classifier...
Measuring Gradient Boosting Classifier...

Classification Computational Cost								
	Task	Model	Training Rows	Testing Rows	Training Time (Seconds)	Prediction Time (Seconds)	Predictions Per Second	Model Size (MB)
0	Classification	Logistic Regression	866870	98698	3.8276	0.1708	578015.85	0.0039
1	Classification	Random Forest Classifier	866870	98698	79.3946	0.2819	350142.58	46.6880
2	Classification	Gradient Boosting Classifier	866870	98698	29.5795	0.2712	363942.40	0.0732

CONFUSION MATRIX

```
1 from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
2 import matplotlib.pyplot as plt
3
4 cm = confusion_matrix(y_test, log_predictions)
5
6 disp = ConfusionMatrixDisplay(
7     confusion_matrix=cm,
8     display_labels=["Delayed (0)", "On Time (1)"]
9 )
10
11 disp.plot(cmap="Blues")
12 plt.title("Logistic Regression Confusion Matrix")
13 plt.show()
```



(where 1e+04 is scientific notation for approximately 10,000)

True Negatives (405) These are trains that were actually delayed, and the model correctly predicted them as delayed.

False Positives (10000) These are trains that were actually delayed, but the model predicted they would be on time. Operationally, these are the most concerning mistakes because passengers and operations would expect an on-time service when a delay actually occurs.

False Negatives (184) These are trains that were actually on time, but the model predicted they would be delayed. These represent false alarms. They may lead to unnecessary operational responses but are generally less serious than missing an actual delay.

True Positives (87838) These are trains that were actually on time, and the model correctly predicted them as on time.

Delayed trains: 7,663 + 8,895 = 16,558